

Situação-problema

Documentação do sistema- Loja Fácil

Ana Laura, Bianca, João P, João V, Lorena, Leonardo S, Maria
Fernanda, Nickolas, Rafael S, Rodrigo, Renan e Thaís.

23 de setembro de 2026

Objetivos

O foco do projeto é melhorar a qualidade do funcionamento da loja do Sr. Carlos do jeito manual em que ele solicitou, para isso, iremos criar um site do zero na qual ele possa ter mais organização e controle no qual deseja, pois a principal dificuldade em que ele relatou foi essa.

E a nossa missão nesse momento é estabelecer alguns objetivos para ser atingidos. Sendo eles:

- Cliente com seu cadastro e deixar ele salvo
- Cadastro do funcionário
- Excluir os cadastros se necessário (Cliente, funcionário ou venda)
- Cadastro dos produtos, os preços, estoque e vendas
- Histórico dos produtos quando vendido, ou quando acabar estoque
- Extrato de venda
- Salvar venda
- Forma de pagamento
- Pessoas com informações
- Listagem dos clientes, funcionários, vendas e produtos

Desejamos entregar nosso site hoje dia 23/09/2026 às 16:00, com as informações e expectativas acima cumpridas.

Com nossos objetivos definidos, o projeto avança para a documentação de requisitos abaixo.

Documento de requisitos

Requisitos funcionais: O que o sistema faz- RF Exemplo: Cadastrar, atualizar dados, excluir produto, realizar login, emitir boleto, gerar relatório de vendas, registrar pagamento, consultar histórico de pedidos, etc)

Requisitos não funcionais: Como o sistema faz- RNF (Exemplo: Responder em até 2 segundos, Suportar até 1.000 usuários simultâneos, garantir disponibilidade de 99.9%, ser executado 24 horas por dia.

Requisitos Funcionais

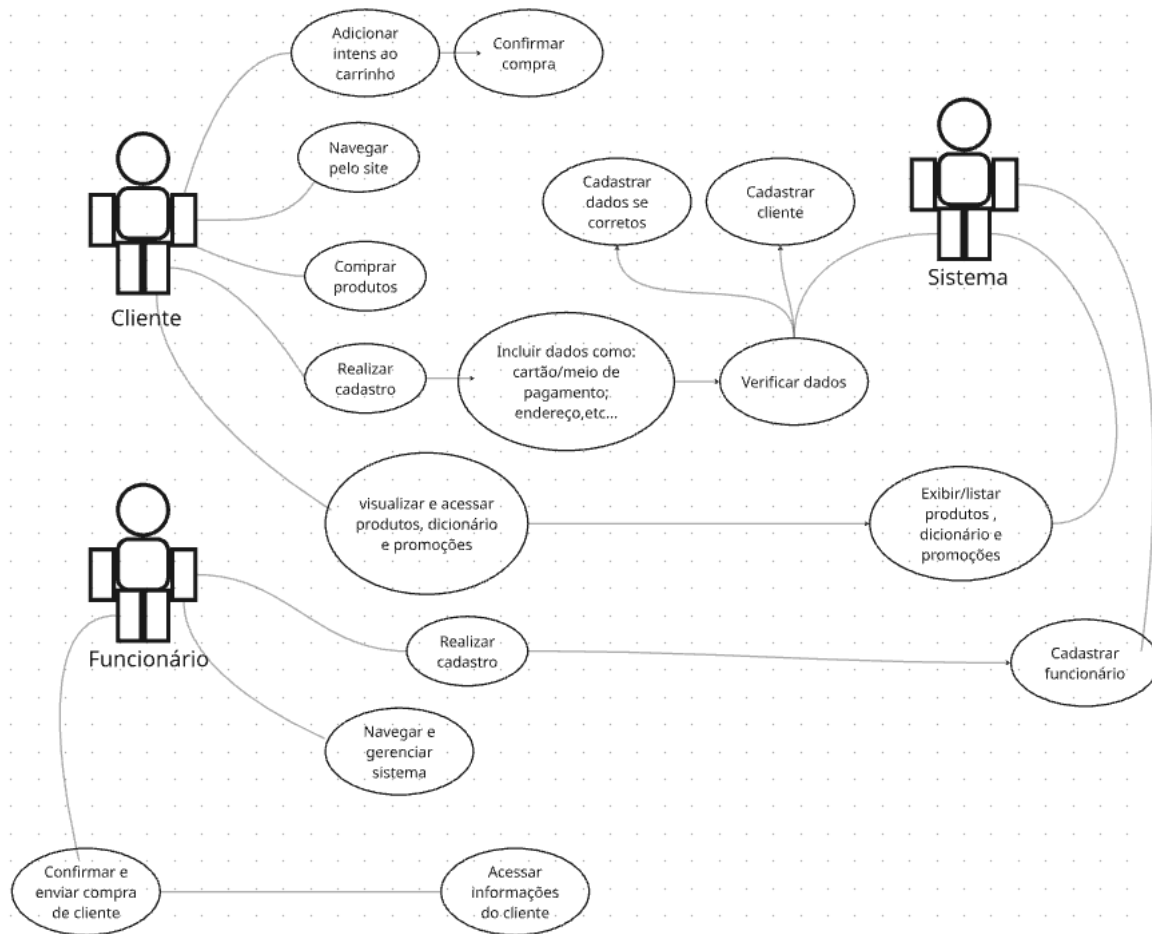
Código	Requisito Funcional	Descrição
RF01	Cadastrar funcionário	Permitir o cadastro de um funcionário.
RF02	Excluir funcionário	Excluir um funcionário cadastrado.
RF03	Listar funcionário	Exibir a lista de funcionários cadastrados.
RF04	Salvar funcionário	Salvar os dados dos funcionários.
RF05	Cadastrar cliente	Permitir o cadastro de um cliente.
RF06	Excluir cliente	Excluir um cliente cadastrado.
RF07	Listar cliente	Exibir a lista de clientes cadastrados.
RF08	Salvar cliente	Salvar as informações dos clientes em arquivos.
RF09	Formulário da venda	Disponibilizar o formulário para realizar vendas.
RF10	Listar vendas	Exibir as vendas que foram realizadas.
RF11	Salvar venda	Salvar os dados das vendas realizadas.

RF12	Listar produtos	O sistema irá permitir visualizar a lista de produtos cadastrados.
RF13	Cadastrar produtos	Permitir o cadastro de produtos.
RF14	Salvar produtos	Salvar os dados dos produtos cadastrados.
RF15	Excluir produtos	Excluir um produto.

Requisitos não funcionais (RNF)

Código	Requisitos não funcionais (RNF)	Descrição
RNF01	Usabilidade	Informações e funcionalidades do sistema organizadas e de fácil acesso.
	Armazenamento	Armazenar os dados em arquivos no servidor.
RNF03	Segurança	Proteger os dados contra acessos e alterações não autorizadas.
RNF04	Manutenção	Facilitar futuras alterações e correções no sistema.
RNF05	Funcionamento local	Funcionar localmente sem depender de serviços externos.
RNF06	Desempenho	Realizar operações de forma rápido e eficiente
RNF07	Escalabilidade	Permitir a inclusão de novos produtos, clientes e funcionalidades no futuro.

Diagrama de Casos de Uso



Fluxograma / Diagrama de Processos

INÍCIO



index.php — Página inicial / Painel de navegação



Abre o menu principal



ESCOLHER UMA FUNÇÃO



CADASTRO DE CLIENTE



cliente.php — Formulário de cadastro específico para clientes



Preencher os dados do cliente



salvar_cliente.php — Processa e grava os dados do cliente



Cliente adicionado à lista



listar_clientes.php — Exibe a lista de clientes cadastrados

CADASTRO DE FUNCIONÁRIO



funcionario.php — Página para cadastrar funcionários



funcionario_form.php — Formulário para preencher os dados do funcionário



Preencher os dados do funcionário



salvar_funcionario.php — Registra os dados do funcionário no sistema



listar_funcionarios.php — Exibe a lista de funcionários cadastrados



Escolher uma ação



— Editar funcionário

|
└─ excluir_funcionario.php — Remove ou desativa o funcionário

CADASTRO DE PRODUTO

↓
produto.php — Página para cadastrar produtos
↓
produto_form.php — Formulário de cadastro de produtos
↓
Preencher nome, preço e quantidade
↓
salvar_produto.php — Salva os dados do produto cadastrado
↓
listar_produto.php — Exibe os produtos cadastrados em forma de lista
↓
Escolher uma ação
↓
└─ excluir_produto.php — Remove o produto do sistema

REALIZAR VENDA

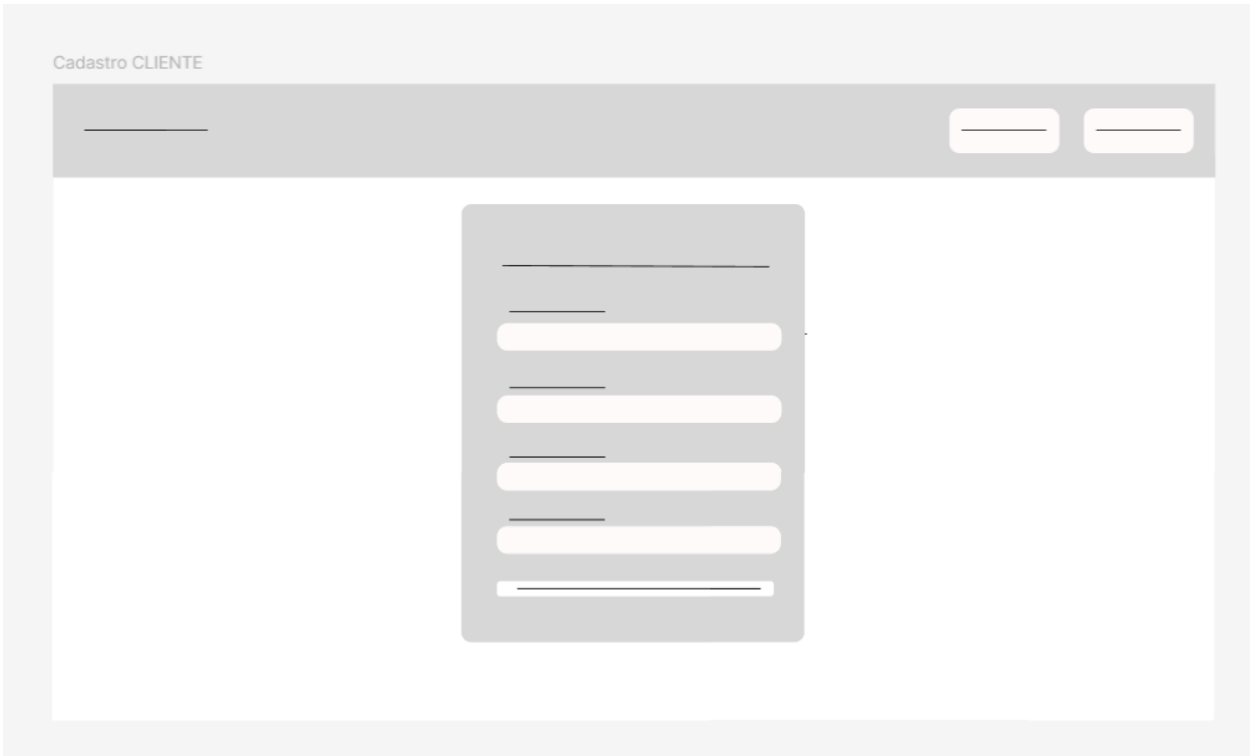
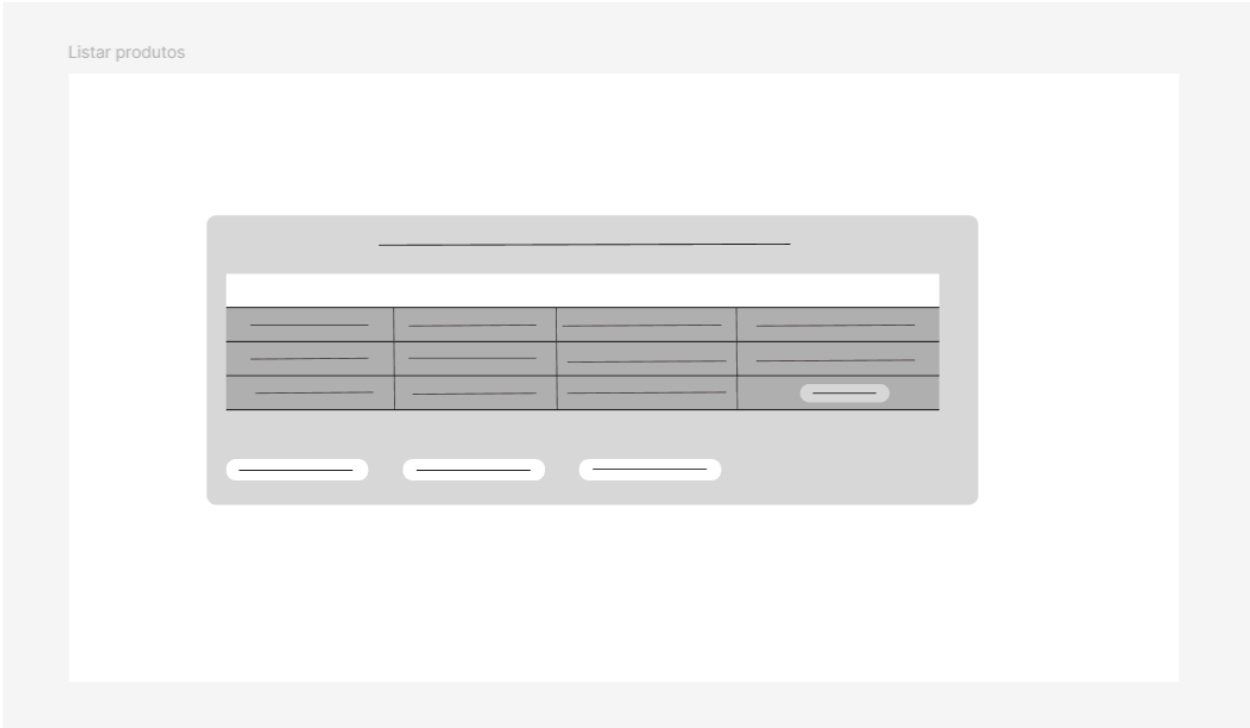
↓
venda_form.php — Formulário para realizar uma venda
↓
Informar cliente, produto, quantidade e valor
↓
venda.php — Processa os dados da venda
↓
salvar_venda.php — Salva os dados da venda
↓
Venda registrada
↓
listar_vendas.php — Exibe as vendas realizadas
↓
extrato.php — Exibe os registros das vendas, com data, produtos e valores
↓

FIM

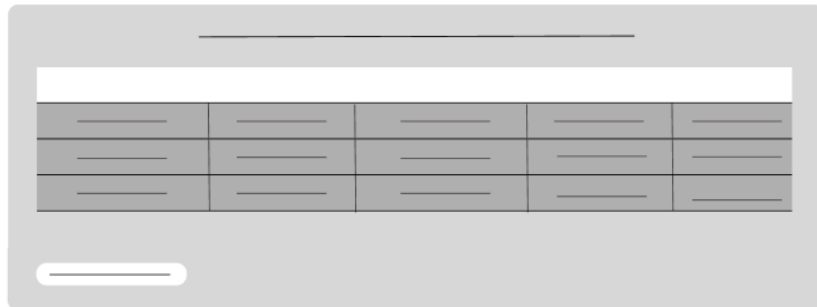
Protótipo das Telas (Wireframe)

Página principal





Ver Vendas



A form for viewing sales data. It features a title bar at the top, a table with 5 columns and 3 rows of data, and a search input field at the bottom left.

Nova Venda



A form for creating a new sale. It includes a title bar and several input fields for data entry.

Ver clientes

Lista de Funcionários

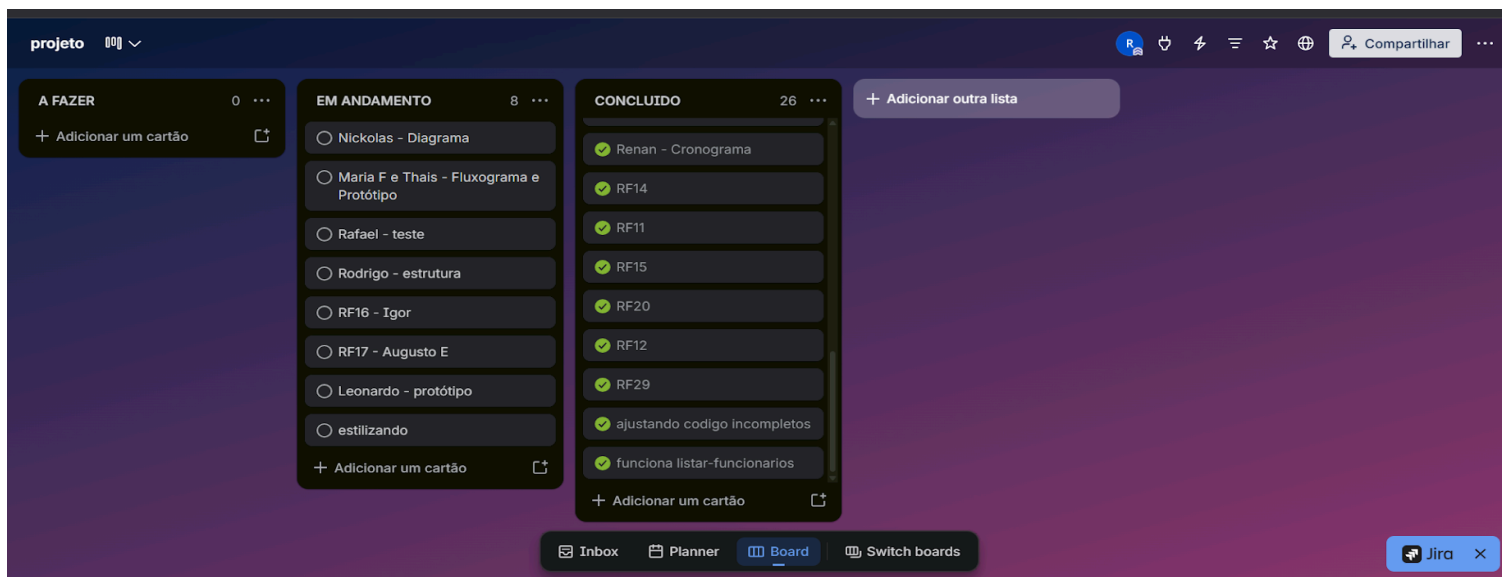
Cadastro FUNCIONÁRIO

Cadastro PRODUTO

Dicionário de Dados / Estrutura do Arquivo

Entidade / Ficheiro	Nome do Campo	Tipo de Dado	Obrigatório?	Exemplo	Descrição
Funcionários	id	Texto	Sim	"funcionário-01"	Identifica um único funcionário
	nome	Texto	Sim	"Carlos Eduardo"	Nome completo do funcionário
	cargo	Texto	Sim	"Gerente"	Função executada na loja (ex: "Caixa", "Gerente").
	login	Texto	Sim	"carlos.loja"	Nome único para identificação
	senha	Texto	Sim	"carlos1234"	Senha única para cadastro
	ativo	Booleano	Sim	TRUE	true (ativo) ou false (desativado/excluído).
Clientes	id	Texto	Sim	"cliente-01"	Identificador um único cliente
	nome	Texto	Sim	"Maria da Silva"	Nome completo do cliente.
	cpf	Texto	Não	"123.456.789-00"	Identificação oficial do cliente
	telefone	Texto	Não	"(19) 99999-8888"	Telefone ou WhatsApp para contato
	endereco	Texto	Não	"Rua das Flores, 123"	Endereço residencial ou comercial do cliente
	ativo	Booleano	Sim	TRUE	Indica se o registo do cliente está ativo
Produtos	id	Texto	Sim	"produto-01"	Identificador de um produto
	codigo_barras	Texto	Não	"7891234567890"	Código na embalagem
	nome	Texto	Sim	"Arroz Tipo 1 - 5kg"	Nome do produto
	categoria	Texto	Sim	"Alimentos"	Categoria para busca
	preco_custo	Decimal	Sim	18.50	Valor pago ao fornecedor
	preco_venda	Decimal	Sim	24.90	Valor cobrado ao cliente
	quantidade_estoque	Inteiro	Sim	45	Quantidade física em estoque
	estoque_minimo	Inteiro	Sim	10	Limite para emitir um alerta para reposição do estoque
Vendas	ativo	Booleano	Sim	TRUE	Diz se está disponível para venda
	id	Texto	Sim	"venda-1001"	Identificador de transação de venda
	id_funcionario	Texto	Sim	"funcionário-01"	ID do funcionário que efetuou a venda
	id_cliente	Texto	Não	"cliente-01"	ID do cliente
	itens	Lista	Sim		Lista dos produtos já vendidos
	valor_total	Decimal	Sim	53.00	Valor total pago
	forma_pagamento	Texto	Sim	"Dinheiro"	Como a compra foi paga
Itens da Venda	data_venda	Data/Hora	Sim	"2026-09-23T09:00:00Z"	Data e hora exatas do fecho da venda
	id_produto	Texto	Sim	"produto-01"	ID do produto selecionado.
	nome_produto	Texto	Sim	"Arroz Tipo 1 - 5kg"	Nome do produto vendido.
	quantidade	Inteiro	Sim	2	Unidades compradas nesta venda.
	preco_unitario	Decimal	Sim	24.90	Preço unitário
	subtotal	Decimal	Sim	49.80	Cálculo total

Cronograma / Kanban de Desenvolvimento



Plano de Testes

	A ▼	B	C
1	db	funcional	não_funcional
2	client-db	x	
3	funcionarios-db	x	
4	prod-db	x	
5	venda-db	x	
6	config.php	x	
7	pessoa.php	x	
8	cliente_form.html	x	
9	funcionario_form.htm	x	
10	index.html	x	
11	index.php	x	
12	produto_form.php	x	
13	venda_form.php	x	

Estrutura

Back-end

Client-db:

- clientes.php:

O código começa incluindo a estrutura da classe base através do arquivo `pessoa.php` e cria a classe `Cliente` estendendo a classe `Pessoa` para aproveitar seus recursos. Em seguida, são declaradas duas variáveis privadas chamadas `crédito` e `saldo`, que servem para guardar as informações financeiras exclusivas do cliente. O programa então executa o método construtor, que recebe os dados de entrada e repassa a identificação, o nome e o CPF para o construtor da classe pai processar, ao mesmo tempo em que armazena o `crédito` e o `saldo` nas variáveis da própria classe. Por fim, o script conta com um método de conversão que coleta os dados herdados da classe pai junto com os valores financeiros internos e transforma tudo em um array organizado, facilitando a leitura e o salvamento das informações no sistema.

`config.php`:

Esse código serve para definir onde os dados dos clientes serão armazenados, ele importa o arquivo `config.php`, define o caminho do arquivo `clientes.json`, guarda esse caminho na variável `$arquivo`, e assim os outros códigos conseguem saber onde encontrar e salvar os dados dos clientes

listar_cliente.php:

fizeram um código que serve para listar os clientes cadastrados no sistema, ele utiliza o arquivo config.php para localizar o arquivo JSON dos clientes, verifica se o arquivo existe, lê os dados dos clientes e transforma o JSON em um array PHP, depois mostra uma lista com o ID, nome, CPF e saldo de cada cliente, caso não exista nenhum arquivo de clientes ele mostra uma mensagem informando que não há clientes cadastrados, e no final cria um link para voltar ao menu principal.

salvar_cliente.php:

fizeram um código que serve para cadastrar um novo cliente no sistema, ele pega os dados preenchidos no formulário, como nome, CPF, crédito e saldo, cria um ID usando o horário atual, cria um novo objeto da classe Cliente, acessa o arquivo JSON onde os clientes são armazenados, lê os clientes que já existem, adiciona o novo cliente à lista, salva novamente os dados no arquivo JSON, e por fim mostra uma mensagem informando que o cliente foi cadastrado com sucesso.

Funcionarios-db

- **config.php:**

Esse código serve para definir o local onde os funcionários serão armazenados, ele importa o arquivo config.php, cria a variável \$funcionariosFile com o caminho do arquivo funcionarios.json, e assim o sistema consegue encontrar e acessar os dados dos funcionários,

- **excluir_funcionario.php:**

Esse código serve para excluir um funcionário do sistema, ele carrega os funcionários armazenados no arquivo JSON, recebe o ID do funcionário enviado pelo formulário, verifica se o ID foi informado, procura o funcionário correspondente e remove ele da lista, depois reorganiza os índices da lista, salva os dados atualizados no arquivo JSON, e por fim redireciona o usuário para a página que mostra os funcionários cadastrados

- **funcionario.php:**

Esta classe funciona como o "molde" ou a estrutura base para representar qualquer funcionário dentro do sistema. Ela aplica boas práticas de programação ao manter as informações sensíveis (como nome, CPF, cargo e salário) protegidas através de variáveis privadas, impedindo alterações indevidas

vinda de fora. O método construtor se encarrega de preparar a pessoa no sistema assim que ela é criada, enquanto os métodos *Getters* e *Setters* controlam a leitura e atualização dos dados. Por fim, as funções `toArray()` e

Esta classe funciona como o "molde" ou a estrutura base para representar qualquer funcionário dentro do sistema. Ela aplica boas práticas de programação ao manter as informações sensíveis (como nome, CPF, cargo e salário) protegidas através de variáveis privadas, impedindo alterações indevidas

`fromArray()` facilitam a conversão dessas informações para formatos leves como JSON, ideais para salvar no banco ou trocar dados com APIs.

- **listar_funcionarios:**

Fizeram o código que começa carregando a classe `Funcionario.php` usando o `require_once`. Depois, o caminho do arquivo de dados é definido na variável `$FuncionariosFile`, que aponta para o arquivo `funcionarios.json`. O sistema verifica se esse arquivo existe e, caso não exista, cria um novo arquivo vazio com um array JSON. Em seguida, o PHP lê o conteúdo do arquivo usando `file_get_contents` e transforma os dados em um array associativo com `json_decode(..., true)`. A partir daí, as informações ficam armazenadas na variável `$funcionarioJSON`.

Depois disso, o programa monta uma tabela (<table>) com uma linha de cabeçalho (<tr>) contendo as colunas: ID, Nome, CPF, Cargo e Salário.

O código também inclui a classe Pessoa através do arquivo pessoa.php e cria a classe Cliente, que herda os recursos da classe Pessoa. Na classe Cliente, existem duas variáveis privadas, credito e saldo, usadas para armazenar as informações financeiras do cliente. O construtor recebe os dados do cliente e envia o ID, nome e CPF para a classe Pessoa, enquanto o crédito e o saldo são armazenados na própria classe. Por fim, o método de conversão junta os dados herdados com as informações financeiras e transforma tudo em um array, facilitando o armazenamento dos dados no sistema.

- **salvar_funcionarios.php:**

Esse código serve para cadastrar um novo funcionário no sistema, ele carrega as configurações e verifica se o arquivo JSON existe, cria o arquivo caso necessário, lê os funcionários já cadastrados, pega os dados enviados pelo formulário como nome, CPF, cargo e salário, verifica se todos os campos obrigatórios foram preenchidos, gera automaticamente um novo ID para o funcionário, cria um objeto da classe Funcionario, adiciona o funcionário à lista, salva os dados novamente no arquivo JSON, mostra uma mensagem de cadastro realizado com sucesso e disponibiliza um link para voltar ao menu principal

prod-db

- **config.php:**

Esse código serve para definir o local onde os produtos serão armazenados, ele importa o arquivo config.php, cria a variável \$produtosFile com o caminho do arquivo produtos.json, e assim o sistema consegue encontrar e acessar os dados dos produtos

- **excluir_produtos.php:**

transformaram o json em array, ler a lista para mostrar o id de valores diferentes, reorganizar o array para o json não ter índices quebrados, atualizar, converter para array novamente e por final redirecionar o usuário para a página de lista.

- **listar_produtos.php:**

O arquivo é iniciado com uma definição de caminho do arquivo (cliente.JSON). É uma variável que serve como uma referência para que o sistema saiba exatamente onde buscar os registro

- **produto.php:**

fizeram uma classe responsável para representar um produto, usaram atributos como ID, nome, preço, criaram um construtor que inicializa os produtos ao criar um novo objeto, criaram um código que retorna o ID, nome, preço do produto, após isso criaram uma public function que altera os valores atribuídos no caso o ID, nome, preço do produto, converteram o objeto para ele ser um array para posteriormente salvar os dados em JSON, criaram um objeto produto para partir de um array, útil para recuperar dados vindos de um arquivo JSON

- **salvar_produto**

venda-db

- **config.php:**

A loja funciona em um fluxo integrado: o index.html é o painel inicial com todos os atalhos, enquanto o config.php garante a leitura, gravação segura dos arquivos JSON e a proteção contra invasões. O cadastro e a proteção de dados da equipe ficam com o funcionario.php e o salvar_funcionario.php. Já nas vendas, o listar_produtos.php exibe a vitrine, o venda_form.php carrega os

clientes e preços em reais para o checkout, e o `salvar_venda.php` fecha a compra, descontado o saldo e gera o extrato.

- **listar_vendas.php**

O arquivo `listar_vendas.php` é a tela que exibe o histórico de pedidos da loja. No topo do código, ele carrega os arquivos JSON de vendas e clientes, criando um mapa rápido para associar o ID de cada comprador ao seu respectivo nome. Na parte visual, ele verifica se existem vendas registradas: se a lista estiver vazia, exibe um aviso simples; caso contrário, monta uma tabela organizada contendo o ID do pedido, o nome do cliente, os produtos comprados agrupados por quantidade, a data da transação e o valor total formatado em reais, encerrando com um botão para retornar ao menu principal.

- **salvar_vendas.php**

O arquivo `salvar_venda.php` processa o envio do formulário de vendas nos bastidores. Ele valida os dados digitados (cliente, produto e quantidade), busca as informações nos arquivos JSON e verifica se o comprador e o item realmente existem. Em seguida, calcula um novo ID automático para o pedido, cria o objeto da venda com a quantidade informada de produtos, grava o registro atualizado na base de dados e redireciona o usuário para a tela de listagem de vendas.

- **venda.php:**

O código cria uma classe chamada `Venda`, ela armazena o cliente, os produtos, o total e a data da venda, permite adicionar produtos à venda também calcula automaticamente o valor total dos produtos, possui métodos para consultar as informações da venda, por fim, permite salvar e recuperar os dados usando JSON

Front-end

- **Venda_form.php:**

o código cria uma página para registrar uma nova venda, ele busca os clientes cadastrados nos arquivos JSON, também busca os produtos

cadastrados, permite escolher o cliente da venda, permite escolher o produto desejado, permite informar a quantidade do produto, envia os dados para `salvar_venda.php`, possui um botão para adicionar o item, possui um link para visualizar as vendas, e organiza tudo em uma página.

- **Cliente _form.html:**

- O documento HTML5 é estruturado de forma a integrar um design moderno em tema escuro com um formulário funcional para coleta de dados de clientes.
- No cabeçalho do arquivo, as tags de configuração definem a codificação UTF-8 para evitar caracteres quebrados e ajustar a visualização para dispositivos móveis. A seção de estilos CSS embutida configura um fundo dinâmico e fixo utilizando gradientes radiais em tom escuro, além de posicionar o menu superior com efeito de desfoque e cantos arredondados nos links de navegação.
- A estrutura visual principal é composta por uma barra de navegação no topo contendo a marca e links para outras áreas do site, seguida por um contêiner centralizado. Dentro deste
- Contêiner, o formulário utiliza o método POST para enviar os dados de forma segura ao arquivo de destino `salvar_cliente.php`. Os campos de entrada de texto para nome e CPF possuem a propriedade de obrigatoriedade ativada, enquanto os campos para crédito e saldo utilizam o tipo numérico com suporte a casas decimais. O processo é finalizado pelo botão de envio, que dispara a transmissão de todas as informações preenchidas pelo usuário.

- **funcionario_form.html:**

O código cria uma página HTML para cadastrar funcionários. Ele possui um formulário para inserir nome, CPF, cargo e salário. Os dados são enviados pelo método POST para o arquivo `salvar_funcionario.php`.

Também possui botões para acessar a página inicial e a lista de produtos. O botão “Listar Funcionários” leva para a página com os funcionários cadastrados.

O CSS é usado para deixar a página organizada e com um visual moderno. Os campos são obrigatórios para evitar o envio de dados vazios.

- **index.html:**

O código cria uma página HTML de redirecionamento automático com suporte visual para o usuário. No cabeçalho do arquivo, a instrução meta refresh configura o navegador para encaminhar o usuário imediatamente (tempo zero) para a página index.php, enquanto a seção de estilos CSS define um visual escuro e utiliza o layout Grid para centralizar o conteúdo perfeitamente no meio da tela. No corpo da página, o script exibe uma mensagem avisando sobre o redirecionamento e fornece um link manual em destaque, garantindo que o usuário consiga acessar a página inicial mesmo se o redirecionamento automático do navegador falhar.

- **index.php:**

O index.html é a página inicial e painel central da aplicação. Ele entrega um layout moderno em tema escuro e organiza o menu de navegação em quatro blocos centrais: Produtos, Vendas, Clientes e Funcionários, disponibilizando atalhos rápidos tanto para os formulários de cadastro quanto para a visualização de cada lista do sistema.

- **produto_form.php:**

O código define uma página web em HTML5 estruturada para o cadastro de produtos em uma loja virtual. No topo, o cabeçalho exibe a marca da loja e um menu fixo com links para a navegação principal. O centro da página contém um formulário de preenchimento onde o usuário insere o nome e o preço do item (com suporte a decimais), enviando esses dados via método POST para o script salvar_produto.php. Logo abaixo do botão de envio, o container disponibiliza links diretos de navegação para consultar a lista de produtos cadastrados ou retornar à tela de vendas. Toda a interface é

renderizada sobre um layout escuro estilizado via CSS interno, com fundo estilizado em padrão pontilhado e caixa centralizada em efeito translúcido.

Estrutura e Função de cada página

ARQUIVO	FUNÇÕES	OBSERVAÇÃO
Index.php	Página inicial / Painel de navegação	Abre o menu principal
pessoa.php	Formulário para cadastrar ou editar dados das pessoas e utilizadores	Registo das informações pessoais
cliente.php	Formulário de cadastro específico para clientes	Capta dados e documenta o cliente
salvar_cliente.php	Processa e grava os dados do cliente	Adiciona para a lista após salvar
excluir_funcionario.php	Faz a remoção ou a desativação de um funcionário.	Apaga o registo de dados do servidor
listar_funcionarios.php	Exibe a lista de todos os funcionários cadastrados	Possui botões de ação para editar ou excluir cada registo
produto.php	Página para cadastrar produtos.	Nome, preço, quantidade
salvar_produto.php	Página para salvar os produtos cadastrados.	Dados do produto
listar_produto.php	Página para visualizar os produtos cadastrados em forma de lista.	Nome, preço e quantidade
venda_form.php	Página com o formulário para realizar uma venda.	Clientes, produtos, quantidade e valor
venda.php	Página responsável por processar a venda.	Dados da venda
salvar_venda.php	Página para salvar os dados da venda.	Cliente, produtos, quantidade e valor.

listar_vendas.php	Página para visualizar as vendas realizadas.	Clientes, produtos, quantidades e valores.
extrato.php	Página para visualizar os registros das vendas.	Data, produtos e valores das vendas.
excluir_produto.php	Página para excluir produtos cadastrados.	Remove o produto do sistema.
produto_form.php	Página com o formulário de cadastro de produtos.	Permite preencher os dados do produto.
funcionario.php	Página para cadastrar funcionários	Permite inserir os dados do funcionário.
salvar_funcionario.php	Página para salvar os funcionários cadastrados.	Registra os dados do funcionário no sistema.
listar_clientes.php	Página para visualizar os clientes cadastrados.	Exibe a lista de clientes registrados.
funcionario_form.php	Página com o formulário de cadastro de funcionários.	Permite preencher os dados do funcionário.

